

Useful Inbuilt Functions in C++

Last Updated : 16 Oct, 2023



In-built functions in C++ are those functions that are part of C++ standard libraries. The purpose of inbuilt functions is to provide common and essential functionality that is frequently required in the programming. In this article, we will look at some of the commonly used inbuilt functions in C++.

1. sqrt() Function

The sqrt() function is used to determine the square root of the value of type double. It is defined inside <cmath> header file.

Syntax

```
sqrt (n)
```

Parameter

- This function takes only one parameter of type double which is a number we want to find the square root of.

Return Type

- The square root of the value of type double.

Example

```
// C++ program to illustrate the sqrt() function
#include <cmath>
#include <iostream>
using namespace std;

int main()
{
    double x = 24;
    double answer;
```

```
    answer = sqrt(x);

    cout << answer << endl;

    return 0;
}
```

Output

```
4.89898
```

2. pow() Function

The `pow()` function is used to find the value of the given number raised to some power. This function is also defined inside `<cmath>` header file.

Syntax

```
double pow(double x, double y);
```

Parameters

- **x**: The base number.
- **y**: The exponential power.

Return Type

- value of **x** raised to the power **y**

Example

```
// C++ program to illustrate the pow() function
#include <cmath>
#include <iostream>
using namespace std;

int main()
{
```

```
int base = 3;
int exponent = 2;

int answer = pow(base, exponent);

cout << answer << endl;
}
```

Output

9

3. sort() Function

The sort() function is part of STL's <algorithm> header. It is a function template that is used to sort the random access containers such as vectors, arrays, etc.

Syntax

```
sort (arr , arr + n, comparator)
```

Parameters

- **arr:** The pointer or iterator to the first element of the array.
- **arr + n:** The pointer to the imaginary element next to the last element of the array.
- **comparator:** The unary predicate function that is used to sort the value in some specific order. The default value of this sorts the array in ascending order.

Return Value

- This function does not return any value.

Example

```
#include <iostream>
#include <algorithm>
```



```
using namespace std;

int main()
{
    int arr[] = { 4, 6, 2 , 8, 7 , 1 , 3};
    int n = sizeof(arr) / sizeof(arr[0]);

    sort(arr, arr + n);

    for (int i = 0; i < n; ++i)
        cout << arr[i] << " ";

    return 0;
}
```

Output

```
1 2 3 4 6 7 8
```

4. find() Function

The find() function is also the part of STL <algorithm> library. This function is used to find some value in the given range. It can be used with both sorted and unsorted datasets as it implements.

Syntax

```
find(startIterator, endIterator, key)
```

Parameter

- **startIterator**: Iterator to the beginning of the range.
- **endIterator**: Iterator to the end of the range.
- **key**: The value to be searched.

Return Value

- If the element is found, then the iterator to the element. Otherwise, iterator to the end.

Example

```
// C++ program to illustrate the find() function
#include <algorithm>
#include <iostream>
#include <vector>

using namespace std;

int main()
{
    vector<int> dataset{ 1, 89, 0, 7, 33, 45 };
    // finding 0 in the above vector
    auto index = find(dataset.begin(), dataset.end(), 0);

    if (index != dataset.end()) {
        cout << "The element found at "
              << index - dataset.begin() << " index";
    }
    else {
        cout << "Element not found";
    }
    return 0;
}
```

Output

```
The element found at 2 index
```

5. binary_search() Function

The `binary_search()` function is also used to find an element in the range but this function implements the binary search instead of the linear search as compared to the `find` function. It is faster than the `find()` function but it can only be implemented on sorted datasets with random access. It is defined inside the `<algorithm>` header file.

Syntax

```
binary_search (starting_pointer , ending_pointer , target);
```

Parameters

- **starting_pointer:** Pointer to the start of the range.
- **ending_pointer:** Pointer to the element after the end of the range.
- **target:** Value to be searched in the dataset.

Return Value

- Returns true if the target is found.
- Else return false.

Example

```
// C++ program to illustrate the binary_search() function
```

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> arr = { 10, 11, 12, 13, 14 };

    // checking if the value 16 is present or not
    if (binary_search(arr.begin(), arr.end(), 16)) {
        cout << 16 << " is present.";
    }
    else {
        cout << 16 << " is not present";
    }

    cout << endl;
}
```

Output

16 is not present

6 . max() Function

The `std::max()` function is used to compare the two numbers and find the maximum among them. It is also defined inside the `<algorithm>` header file.

Syntax

```
max ( a , b )
```

Parameters

- **a**: First number
- **b**: Second number

Return Value

- This function returns the larger number among the two numbers **a** and **b**.
- If the two numbers are equal, it returns the first number.

Example

```
// C++ program to illustrate the max() function
#include <algorithm>
#include <iostream>
using namespace std;

int main()
{
    cout << max(7, 6);

    return 0;
}
```

Output

7 . min() Function

The `std::min()` function is used to compare the two numbers and find the minimum among them. It is also defined inside the `<algorithm>` header file.

Syntax

```
min (a , b)
```

Parameters

- **a:** First number
- **b:** Second number

Return Value

- This function returns the smaller number among the two numbers **a** and **b**.
- If the two numbers are equal, it returns the first number.

Example

```
// C++ program to illustrate the min() function
#include <algorithm>
#include <iostream>
using namespace std;

int main()
{
    cout << min(3, 4);

    return 0;
}
```

Output

3

8. swap() Function

The `std::swap()` function provides the basic functionality to swap two values. It is defined inside `<algorithm>` header file.

Syntax

```
swap(a , b);
```

Parameters

- **a**: First number
- **b**: Second number

Return Value

- This function does not return any value.

Example

```
// C++ program to illustrate the swap() function
#include <algorithm>
#include <iostream>

using namespace std;

int main()
{
    int a = 3, b = 4;
    cout << "Before swap: " << endl;
    cout << "a: " << a << " b: " << b;
    cout << endl;

    // using swap
    swap(a, b);

    cout << "After swap: " << endl;
    cout << "a: " << a << " b: " << b;

    return 0;
```

```
}
```

Output

```
Before swap:  
a: 3 b: 4  
After swap:  
a: 4 b: 3
```

9. tolower() Function

The `tolower()` function is used to convert the given alphabet character to the lowercase. It is defined inside the `<cctype>` header.

Syntax

```
tolower (c);
```

Parameters

- **c**: The alphabet to be converted.

Return Value

- Lowercase of the character `c`.
- Returns `c` if `c` is not an alphabet.

Example

```
// C++ program to illustrate the use of tolower()  
#include <cctype>  
#include <iostream>  
using namespace std;  
  
int main()  
{  
  
    string str = "GFG";
```



```
// using tolower() for each character
for (auto& a : str) {
    a = tolower(a);
}

cout << str;
return 0;
}
```

Output

gfg

10. toupper() Function

The toupper() function is used to convert the given alphabet character to uppercase. It is defined inside the <cctype> header.

Syntax

```
toupper (c);
```

Parameters

- **c**: The alphabet to be converted.

Return Value

- Uppercase of the character c.
- Returns c if c is not an alphabet.

Example

```
// C++ program to illustrate the use of toupper()
#include <cctype>
#include <iostream>
using namespace std;

int main()
{
```

```
string str = "geeksforgeeks";

// using toupper() for each character
for (auto& a : str) {
    a = toupper(a);
}

cout << str;
return 0;
}
```

Output

GEEKSFORGEEEKS

 Comment

More info 

Advertise with us

Similar Reads

C++ Programming Language

C++ is a programming language known for its fast speed, low level memory management and is often taught as first programming language. Why Learn C++?C++ is often taught as a foundational language ...

 5 min read

C++ Overview



C++ Basics



C++ Variables and Constants



C++ Data Types and Literals



C++ Operators



C++ Input/Output



C++ Control Statements



C++ Functions



C++ Pointers and References



Do Not Sell or Share My Personal Information